

AI-Augmented Mutation Testing Framework

چارچوب آزمون جهش مبتنی بر هوش مصنوعی

From Intelligent Test Generation to Mathematical Optimization

Motivation: Why Mutation Testing?

Mutation Testing Flow



- پوشش کد بالا (Code Coverage) به معنای کیفیت بالای تست نیست.
- بسیاری از تست‌ها اجرا می‌شوند، اما هیچ خطای واقعی را آشکار نمی‌کنند.
- آزمون جهش (Mutation Testing) یکی از دقیق‌ترین معیارهای سنجش قدرت واقعی مجموعه تست است.
- هدف: اندازه‌گیری توان تست‌ها در تشخیص خطاهای منطقی، نه فقط اجرای خطوط کد.



Mutation Score Formula

$$\text{Mutation Score} = (\text{Killed Mutants}) \div (\text{Total Non-Equivalent Mutants}) \times 100\%$$

نکته مهم: جهش‌یافته‌های هم‌ارز (Equivalent Mutants) باید از مخرج حذف شوند.

 تولید تست مؤثر

- دشواری تولید تست هدفمند و مؤثر

 هزینه اجرا

- هزینه بالای اجرای تست روی هر Mutant

 انفجار ترکیبیاتی

- تعداد بسیار زیاد جهش‌یافته‌ها با رشد کد

 مقیاس صنعتی

- هزینه زمانی/پرداشی در مقیاس بزرگ

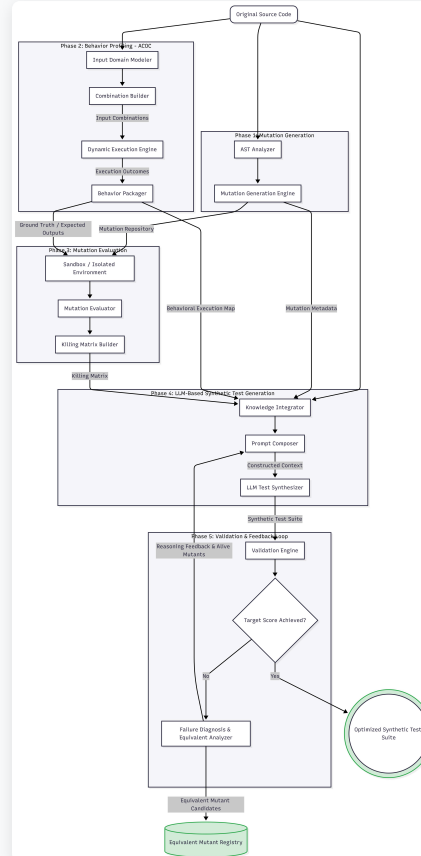
 کمینه‌سازی

- یافتن کمینه‌ترین مجموعه تست

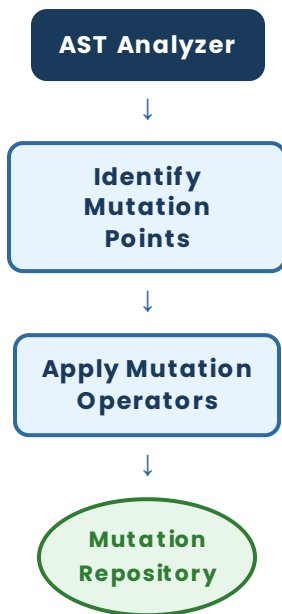
 هم‌ارزی

- شناسایی جهش‌یافته‌های هم‌ارز

سؤال اصلی پروژه: چگونه می‌توان با کمترین تعداد تست، بالاترین Mutation Score را به‌دست آورد؟



5-Phase AI-driven Pipeline



- تبدیل کد به درخت نحو (AST) — دقت بالاتر از دستکاری انتزاعی متنی.
- شناسایی تمام نقاط قابل جهش در کد.
- اعمال عملگرهای جهش (Mutation Operators) به صورت جامع.
- ذخیره مستقل هر جهش یافته در مخزن برای ارزیابی مستقل.

خروجی: مخزن کامل جهش یافته‌ها (Complete Mutation Repository)

دسته (Category)	Example Operator	توضیح
Arithmetic	$+$ $\rightarrow$ $-$	جایگزینی عملگرهای ریاضی
Relational	$<$ $\rightarrow$ $<=$	جایگزینی عملگرهای مقایسه‌ای
Logical	AND $\rightarrow$ OR	جایگزینی عملگرهای منطقی
Assignment	$+=$ $\rightarrow$ $-=$	جایگزینی عملگرهای انتساب
Unary	- حذف / افزودن	تغییر عملگرهای یک‌متغیره

همه جهش‌یافته‌های تولیدشده در مخزن ذخیره و تعداد هر دسته در گزارش ثبت می‌شود.

در این معماری، ACOC صرفاً تولیدکننده تست نیست؛ بلکه نقش استخراج کننده دانش رفتاری برنامه را ایفا می کند.



- تعریف دامنه ورودی و افراز هم ارزی (Equivalence Partitioning) .
- اعمال ضرب دکارتی (Cartesian Product) برای پوشش تمام ترکیب ها.
- اجرای پویا (Dynamic Execution) روی تمام ترکیب ها و ثبت رفتار واقعی.
- خروجی: نقشه رفتاری اجرا (Behavioral Execution Map) — مرجع صحت برای فاز ۴.

تعاریف کلیدی

- افراز هم‌ارزی (Equivalence Partitioning) : تقسیم دامنه ورودی به کلاس‌های نماینده.
- مقادیر مرزی (Boundary Values) : شناسایی نقاط حساس دامنه.

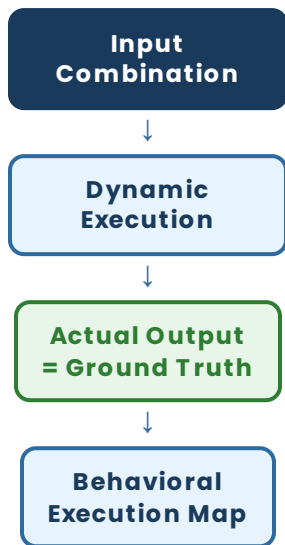
age مثال عملی - پارامتر

مقدار نامعتبر ← -1

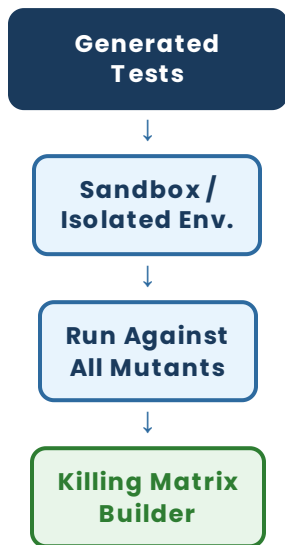
مرز پایین ← 0

مرز معنادار ← 18

مرز بالا ← 65



- تمام ترکیب‌های ورودی به صورت پویا روی برنامه اصلی اجرا می‌شوند.
- خروجی واقعی هر **Ground Truth /** ذخیره بدون تعریف **Oracle** می‌شود دستی اجرا به عنوان
- این اطلاعات در فاز ۴ مستقیماً به مدل زبانی منتقل می‌شوند.
- هیچ مدل زبانی‌ای نمی‌تواند این دانش را بدون اجرای واقعی کد با این دقت تولید کند.



- تست‌های تولیدشده از فاز ۲ روی تمام جهش‌یافته‌ها اجرا می‌شوند.
- هر اجرا در یک **(Sandbox)** محیط ایزوله انجام می‌شود تا از تداخل اجراها جلوگیری شود.
- نتایج جمع‌آوری و در ماتریس کشتار ثبت می‌شوند.
- خروجی نهایی: **Killing Matrix** — پایه همه تصمیمات بعدی.

Test	M1	M2	M3
T1	✓	✗	✓
T2	✗	✓	✗
T3	✓	✓	✗

- سطرها = تست‌ها، ستون‌ها = جهش‌یافته‌ها.
- ✓ یعنی آن تست، آن جهش‌یافته را کشته است.
- ✗ یعنی نتوانسته آن را شناسایی کند.
- در این نمونه: هیچ جهش‌یافته‌ای زنده نمانده.

در موتور اول: این ماتریس به مدل زبانی داده می‌شود. در موتور دوم: مستقیماً وارد مسئله بهینه‌سازی می‌شود.

ورودی‌های مدل زبانی

- Source Code — کد منبع اصلی
- Mutants — مجموعه کامل جهش‌یافته‌ها
- Killing Matrix — ماتریس کشتار
- Behavior Profile — نقشه رفتاری اجرا

خروجی‌های مدل زبانی

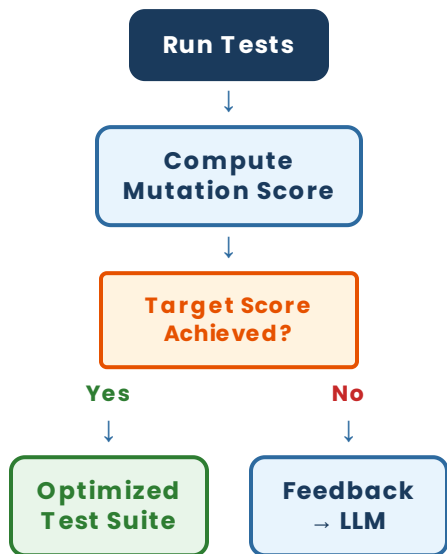
- New Tests — تست‌های کاملاً جدید
- Condensed Suite — مجموعه تست متراکم
- New Assertions — ادعاهای جدید
- New Inputs — ورودی‌های ابداعی برای جهش‌های سرسخت

روش های کلاسیک

- تولید تعداد زیاد تست
- گزینش بهترین زیرمجموعه با Greedy / ILP
- محدود به تست های از پیش موجود
- نتیجه هرگز بهتر از تست های اولیه نیست ❌

روش پیشنهادی ما

- تحلیل رفتاری کامل برنامه
- تولید تست کمینه از ابتدا
- بدون مرحله گزینش ✓
- ترکیب مستقیم توسط مدل زبانی ✓



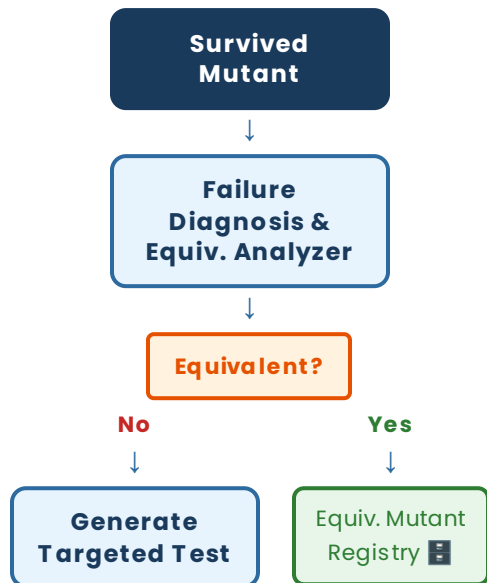
- تست‌های تولیدشده روی تمام جهش‌یافته‌ها اجرا می‌شوند.
- امتیاز جهش (Mutation Score) جدید محاسبه می‌شود.
- اگر هدف حاصل شد: پایان
Optimized Synthetic Test Suite
- اگر نه: بازخورد به مدل زبانی → تولید تست هدفمند جدید.

▪ جهش‌یافته‌ای که هیچ تستی نمی‌تواند آن را بکشد → دو یا هم‌ارزی
احتمال: ضعف تست واقعی.

▪ مدل زبانی برای تحلیل دلیل زنده ماندن فراخوانی می‌شود.

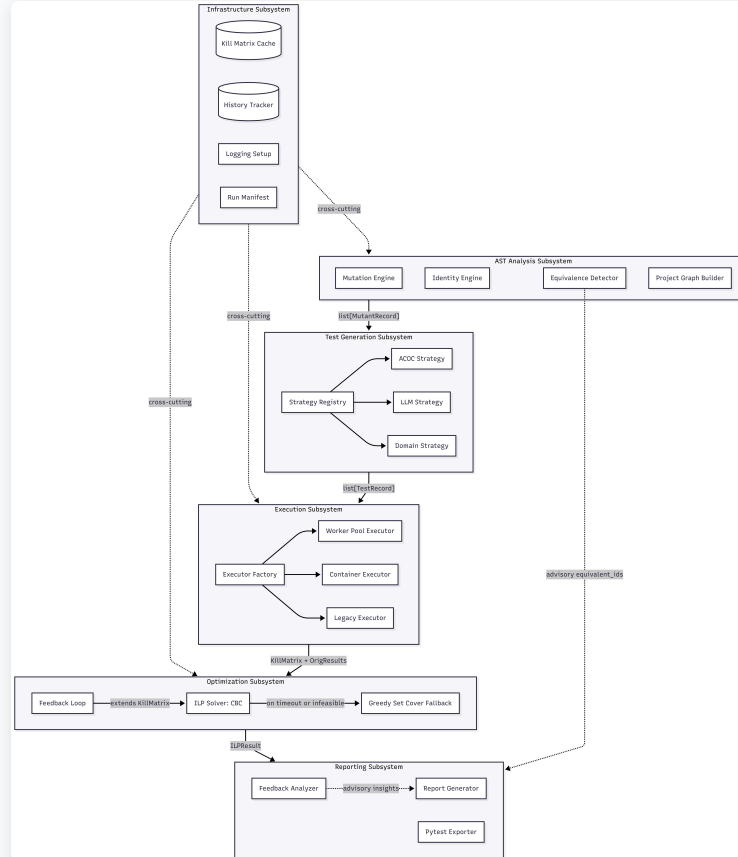
▪ اگر هم‌ارز نیست: تست هدفمند جدید → بازگشت به فاز ۴.

▪ اگر هم‌ارز است: حذف از محاسبه امتیاز + ثبت در مخزن.



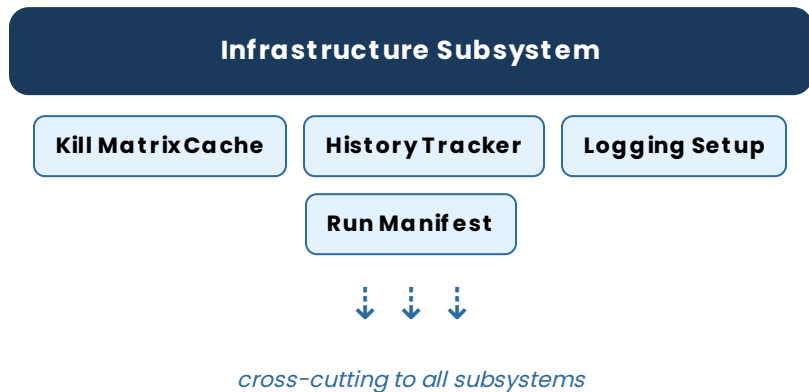
قابلیت	Capability	Status
تولید تست مبتنی بر هوش مصنوعی	AI-driven Test Generation	✓
مبتنی بر رفتار واقعی برنامه	Behavior-aware	✓
تولید از ابتدا (بدون گزینش)	Generate from Scratch	✓
حلقه بازخورد خودکار	Automatic Feedback Loop	✓
شناسایی جهش‌یافته‌های هم‌ارز	Equivalent Mutant Detection	✓

محدودیت اساسی: مدل زبانی ذاتاً **Black-box** است — هیچ تضمین ریاضی برای بهینگی سراسری وجود ندارد.



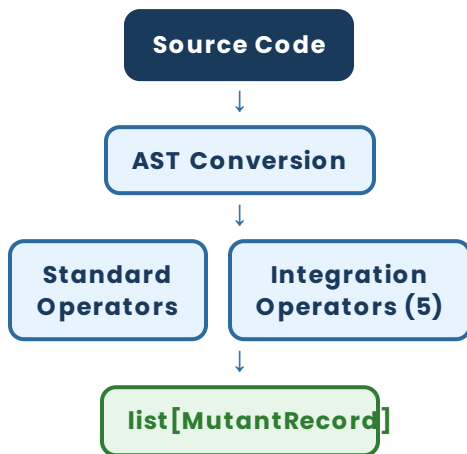
Subsystem-Based Architecture with Exact Mathematical Optimization

- موتور اول کاملاً مبتنی بر مدل زبانی است.
 - مدل‌های زبانی ذاتاً **Black-box** هستند.
 - هیچ تضمین ریاضی برای کمینه‌بودن مجموعه تست در موتور اول وجود ندارد.
 - نیاز به یک چارچوب مستقل برای اعتبارسنجی و بنچمارک (**Validation / Benchmark**) .
- موتور دوم، معیار سنجش می‌سازد، نه فقط یک راه‌حل جایگزین.



- **Kill Matrix** — ذخیره‌سازی نتایج برای جلوگیری از اجرای **Cache** تکراری.
- **History Tracker** — ثبت تاریخچه کامل اجراها.
- **Logging Setup** — لاگ‌گیری یکپارچه در سطح کل پروژه.
- **Run Manifest** — مانیفست نسخه‌دار هر اجرا برای بازتولیدپذیری.

این چهار مؤلفه به صورت **Cross-cutting** روی **AST Analysis**، **Test**، **Generation**، **Execution** و **Optimization** اثر می‌گذارند.



- تبدیل کد به **AST** — دقت بالاتر نسبت به دستکاری متنی.
- اسکن تمام نقاط قابل جهش: عملگرهای ریاضی، رابطه‌ای، منطقی.
- عملگرهای یکپارچگی **(Integration Operators)** — مخصوص نقص‌های ۵ نوع
- هر عملگر به فهرستی از حالت‌ها نگاشت می‌شود (نه یک حالت واحد).

AST Analysis Subsystem:

Mutation Engine | Identity Engine | Equivalence Detector | Project Graph Builder

نظریه اثر هم‌نشینی

- باگ‌های پیچیده، ترکیبی از باگ‌های ساده‌اند.
- اگر تست‌ها جهش‌های مرتبه اول را شکار کنند، خطاهای ترکیبی را هم ضمناً پوشش می‌دهند.
- صرفه‌جویی چشمگیر در منابع محاسباتی.

مقایسه رویکردها

✓ **First-Order Mutants**

Single change per mutant

Used in this project✗ **Higher-Order Mutants**

Multiple changes — combinatorial explosion

 LLM Strategy

- تفاوت خط اصلی/جهش به مدل داده می شود
- دسته بندی جهش ها برای کاهش هزینه **API**
- یک نماینده از هر دسته ارسال می شود

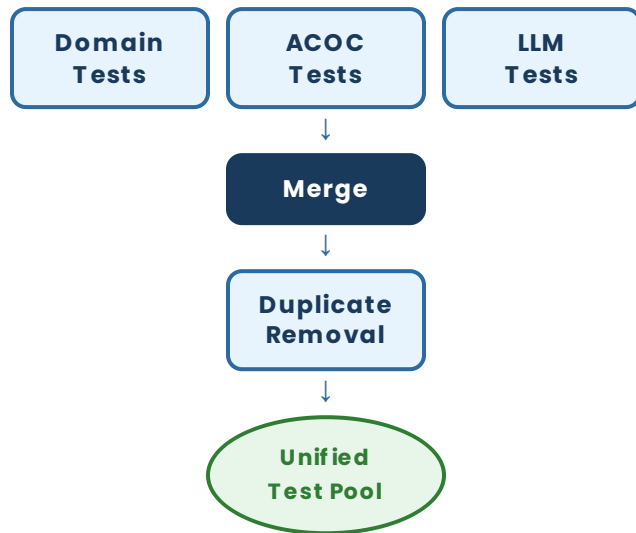
 ACOC Strategy

- افراز هم ارزی + ضرب دکارتی
- سقف ترکیببایاتی برای کنترل تعداد
- نمونه گیری با بذر ثابت (**Fixed Seed**)

 Domain Strategy

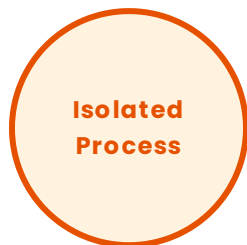
- حالات مرزی و خاص
- لیست خالی، مقادیر منفی
- تست های حساسیت مرزی

مدیریت هر سه استراتژی توسط Strategy Registry — خروجی: `list[TestRecord]`



- سه منبع تست (Domain + ACOC + LLM) با هم ادغام می شوند.
- موارد تکراری شناسایی و حذف می شوند (Duplicate Removal).
- نتیجه: استخر یکتا و ترکیبی از تست ها (Unified Test Pool).

یکتاسازی از اجرای بی فایده تست های تکراری روی جهش یافته ها جلوگیری می کند.



Resource Limits

Wall-clock Timeout

Adaptive Timeout

- اجرای هر جهش یافته در یک پروسه مجزا (Process Isolation).
- سه لایه سقف (RAM/CPU) + زمان سنج دیوار-ساعتی + حفاظتی: منابع زمان سنج تطبیقی هوشمند.
- اجرای کاملاً موازی توسط Worker Pool Executor.

Executor Factory manages:

- **Worker Pool Executor** — parallel execution
- **Container Executor** — strong isolation
- **Legacy Executor** — compatibility mode

Test	M1	M2	M3	M4
T1	✓	✗	✗	✓
T2	✗	✓	✓	✗

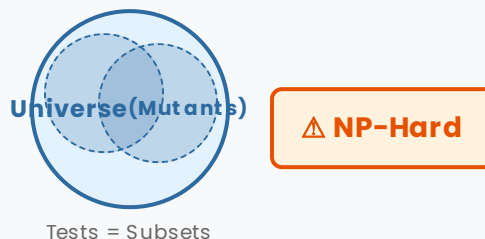
▪ مفهوم مشابه موتور اول: سطر=تست، ستون=جهش یافته.

▪ خروجی دقیق زیرسیستم اجرا: **KillMatrix + OrigResults**.

▪ **OrigResults** برای محاسبه زمان سنج تطبیقی و رفتار خط مبنا لازم است.

نکته فنی مهم: **OrigResults** = نتایج اجرای کد سالم — پایه مقایسه برای تشخیص رفتار متفاوت جهش یافته.

- هدف: یافتن کمینه‌ترین مجموعه تست با حفظ همان قدرت پوشش جهش.
- این دقیقاً مسئله کلاسیک **Set Cover** است.
- این مسئله از نظر محاسباتی **NP-Hard** است.



Universe = جهش‌یافته‌ها | Subsets = تست‌ها | هدف: کمترین تعداد Subset برای پوشش کل Universe

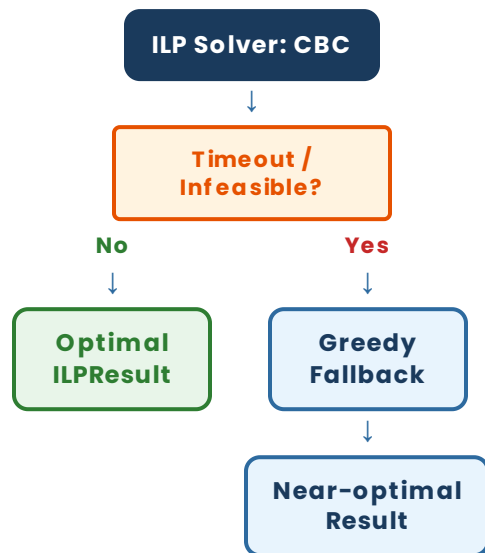
- برای هر تست i ، یک متغیر بودویی $x_i \in \{0, 1\}$ تعریف می‌شود.
- اگر $x_i = 1$ یعنی تست انتخاب شده؛ $x_i = 0$ یعنی انتخاب نشده.
- حل‌گر: CBC (Coin-or Branch and Cut) — متن‌باز و قدرتمند.

```
// Objective: minimize total selected tests
Minimize:   $\sum x_i$       for  $i = 1..n$ 

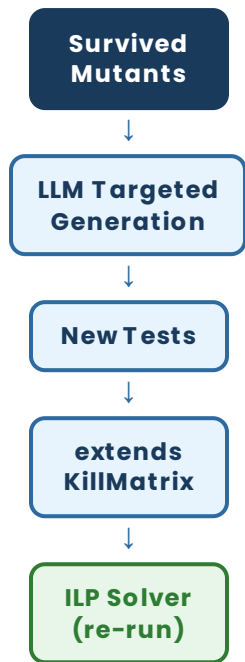
// Coverage constraint: every mutant must be killed
Subject to:  $\sum x_i \geq 1$   for each mutant  $m$  (over tests covering  $m$ )

 $x_i \in \{0, 1\}$ 
```

اگر CBC مسئله را حل کند: پاسخ تضمین‌شده بهینه سراسری — این قطعیت ریاضی در موتور اول ممکن نیست.



- اگر حل گر ILP در زمان معقول جواب ندهد (Timeout) ...
- ... یا مسئله ناشدنی باشد (Infeasible) ...
- سیستم به الگوریتم حریصانه (Greedy Set Cover) عقب نشینی می کند.
- سرعت بالا — بدون تضمین بهینگی دقیق، اما همیشه یک خروجی می دهد.



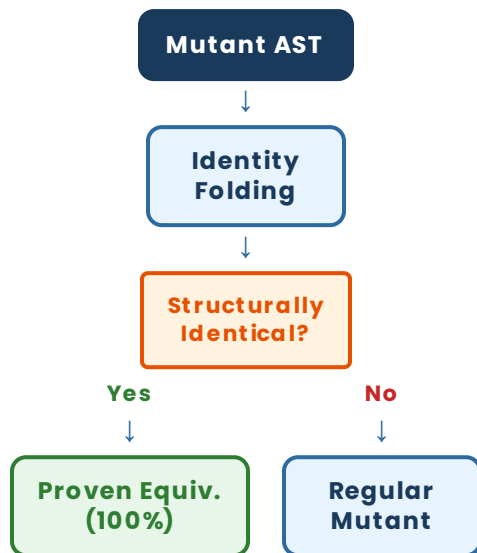
- شناسایی جهش‌یافته‌های هنوز زنده‌مانده پس از حل ILP/Greedy.
- فراخوانی مجدد مدل زبانی — با دانستن این‌که ورودی‌های قبلی ناکام بوده‌اند.
- تولید ورودی کاملاً جدید و هدفمند.
- افزودن تست جدید به ماتریس کشتار: **Feedback Loop extends KillMatrix**.
- اجرای دوباره فرآیند ILP روی ماتریس گسترش‌یافته.

- جهش‌یافته هم‌ارز: ساختار تغییر کرده اما رفتار دقیقاً مثل کد اصلی → غیرقابل کشتن.

- تکنیک **Identity Folding**: ساده‌سازی ریاضی کد جهش‌یافته **AST** روی

- اگر ساختار ساده‌شده دقیقاً با کد اصلی یکی شد: اثبات قطعی هم‌ارزی (۱۰۰٪).

- خروجی: **advisory** — بینش مشورتی، بدون دستکاری **equivalent_ids** امتیاز.



نوع امتیاز	Score Type	تعریف و ماهیت
امتیاز خام	Raw Score	نسبت کشته شده ها به کل — بدبینانه ترین تخمین
امتیاز تعدیل شده	Adjusted Score	با فرض هم ارز بودن همه زنده مانده ها — خوش بینانه ترین حالت
امتیاز هم ارزی آگاه	Equivalent-aware Score	با حذف فقط موارد اثبات شده توسط Identity Engine — واقع گرایانه ترین

این رویکرد سه گانه، شفافیت کامل به تحلیل گر می دهد تا بسته به نیاز از هر معیار استفاده کند.

قابلیت	Capability	Status
تحلیل AST + عملگرهای یکپارچگی	AST-based Mutation + Integration Operators	✓
بهینه‌سازی دقیق مبتنی بر ILP	ILP-based Exact Optimization	✓
اجرای موازی و کاملاً ایزوله	Parallel & Sandboxed Execution	✓
مکانیزم پشتیبان حریصانه	Greedy Fallback Mechanism	✓
تشخیص قطعی هم‌ارزی با تاکردن اتحاد	Equiv. Detection (Identity Folding)	✓

این پنج ویژگی، دقیقاً چیزهایی هستند که موتور اول به دلیل ماهیت Black-box نمی‌تواند با همین سطح از قطعیت ارائه دهد.

 موتور دوم

- بهینه‌سازی ریاضی دقیق
- اعتبارسنجی صوری
- پوشش مجموعه بهینه با تضمین ریاضی



Validation

Benchmark

Feedback

 موتور اول

- مبتنی بر هوش مصنوعی
- یادگیری رفتاری از اجرای واقعی
- ترکیب مستقیم تست از ابتدا

ویژگی	Engine I	Engine II
تولید تست	✓	✓
استفاده از مدل زبانی	برای تولید مستقیم تست ✓	فقط برای موارد ناتمام ✓
نوع بهینه‌سازی	Implicit (در دل LLM)	Explicit ILP
بهینگی سراسری	؟ نامعلوم	تضمین شده ✓
اعتبارسنجی صوری	—	✓
تشخیص هم‌ارزی	LLM استدلال	AST اثبات ریاضی

- طراحی چارچوب هوشمند برای تولید مستقیم مجموعه تست کمینه با استفاده از LLM .
- بازتعریف نقش ACOC : استخراج‌کننده دانش رفتاری، نه تولیدکننده نهایی تست.
- طراحی موتور مستقل مبتنی بر بهینه‌سازی ریاضی با تضمین صوری .
- ارائه سازوکار تشخیص جهش‌یافته‌های هم‌ارز با ترکیب تحلیل AST و استدلال هوشمند.
- استفاده از موتور دوم به‌عنوان معیار اعتبارسنجی و بنچمارک برای موتور اول.
- معماری مکمل با پتانسیل توسعه به پژوهش‌ها و مقالات مستقل.

AI for Creativity, Mathematics for Certainty.

هوش مصنوعی برای خلاقیت، ریاضیات برای اطمینان.

دو موتور، یک هدف مشترک — مجموعه تستی کوچک، قوی، و قابل اعتماد.

Slide 37 / 38 AI-Augmented Mutation Testing Framework

Thank You

با سپاس از توجه شما

Questions?

اطلاعات تماس / مخزن کد پروژه

یادداشت‌های گویا — Speaker Notes Appendix

AI-Augmented Mutation Testing Framework — 38 Slides

سلام و وقت بخیر. در این ارائه، یک چارچوب کامل برای آزمون جهش مبتنی بر هوش مصنوعی را معرفی می‌کنیم. این پروژه شامل دو موتور مکمل است: موتور اول که با کمک مدل‌های زبانی بزرگ، مجموعه تست را به صورت هوشمند و مستقیم تولید می‌کند، و موتور دوم که با بهینه‌سازی ریاضی دقیق، همان مسئله را از زاویه‌ای کاملاً متفاوت و قابل اثبات حل می‌کند.

سؤال اصلی این است: وقتی می‌بینیم Coverage یک تست سویییت ۹۵ درصد است، آیا واقعاً می‌توانیم به آن اعتماد کنیم؟ ممکن است یک تست فقط یک خط کد را اجرا کند، بدون اینکه نتیجه آن خط را واقعاً بررسی کند. Mutation Testing این مشکل را حل می‌کند: یک نسخه کوچک و خراب از برنامه می‌سازیم — مثلاً یک عملگر جمع را به تفریق تبدیل می‌کنیم — و می‌بینیم آیا تست‌های موجود می‌توانند این خرابی را تشخیص دهند یا نه.

گردش کار سنتی آزمون جهش از چهار مرحله تشکیل شده. نکته مهم در فرمول این است که جهش یافده‌های هم‌ارز (Equivalent) باید از مخرج کسر حذف شوند، چون اصولاً قابل کشتن نیستند و وارد کردن آن‌ها، امتیاز را به ناحق پایین می‌آورد.

این شش چالش، اصلی‌ترین موانع پیاده‌سازی عملی آزمون جهش در مقیاس واقعی هستند. در ادامه نشان می‌دهیم که موتور اول و موتور دوم ما، هرکدام از زاویه‌ای متفاوت، دقیقاً به این شش چالش پاسخ می‌دهند.

اسلاید ۵ — معماری موتور اول

این تصویر، نقشه کامل معماری موتور اول است. از کد منبع اصلی دو مسیر موازی شروع می‌شود: یکی تحلیل ساختاری کد برای تولید جهش‌یافته‌ها، و دیگری پروفایل‌برداری رفتاری برنامه با تکنیک ACOC. این دو مسیر در فاز سوم با هم تلاقی می‌کنند و Killing Matrix می‌سازند.

کد منبع به درخت نحو انتزاعی یا AST تبدیل می‌شود، چون AST ساختار واقعی برنامه را همان‌طور که کامپایلر می‌بیند نشان می‌دهد. این دقت بسیار بالاتری نسبت به دستکاری متنی کد فراهم می‌کند.

این جدول دسته‌بندی اصلی عملگرهای جهش را نشان می‌دهد. نکته مهمی که در موتور دوم اضافه می‌شود، عملگرهای یکپارچگی است که مخصوص نقص‌های سطح کنترل برنامه طراحی شده‌اند.

ACOC در روش‌های سنتی مستقیماً به عنوان مجموعه تست نهایی استفاده می‌شود. اما در معماری ما، نقش ACOC کاملاً متفاوت تعریف شده: ACOC فقط برای استخراج دانش رفتاری برنامه به کار می‌رود، نه تولید تست نهایی.

اسلايد ۹ — مدل سازی ورودی

با دو تکنیک کلاسیک تست نرم افزار دامنه ورودی تعریف می شود: افراز هم ارزی و تحلیل مقادیر مرزی. برای مثال، اگر پارامتر ورودی سن باشد، مقادیر نماینده می توانند منفی یک، صفر، هجده و شصت و پنج باشند.

نکته مهم: Ground Truth به صورت پویا استخراج می‌شود، نه تعریف دستی. تابع اصلی برنامه روی تمام ترکیب‌های تولیدشده اجرا می‌شود و خروجی واقعی آن به عنوان مرجع صحت ذخیره می‌شود. این روش خطای انسانی در تعریف اوراکل را حذف می‌کند.

چون اجرای کد جهش یافته می تواند رفتارهای غیرمنتظره داشته باشد، هر اجرا در یک محیط ایزوله انجام می شود تا یک Mutant خراب نتواند روی نتیجه Mutant دیگر اثر بگذارد.

این ماتریس، پایه و اساس تمام تصمیم‌گیری‌های بعدی در هر دو موتور است. در موتور اول، این اطلاعات به مدل زبانی داده می‌شود؛ در موتور دوم، همین ماتریس مستقیماً وارد مسئله بهینه‌سازی ریاضی می‌شود.

این فاز، قلب اصلی و مهم‌ترین نوآوری موتور اول است. به جای انتخاب از میان تست‌های موجود، تمام دانش جمع‌آوری‌شده در اختیار یک مدل زبانی بزرگ قرار می‌گیرد تا مجموعه‌ای کاملاً تازه و از ابتدا بسازد. مدل می‌تواند برای Mutant هایی که با هیچ تست موجودی کشته نشده‌اند، ورودی‌های کاملاً جدید ابداع کند.

این اسلاید تفاوت بنیادین معماری ما با تمام روش‌های متداول را خلاصه می‌کند. در روش‌های کلاسیک، نتیجه نهایی هرگز نمی‌تواند بهتر از بهترین تست‌هایی باشد که از قبل وجود داشته‌اند. اما در روش ما، اصلاً مرحله انتخاب وجود ندارد.

تست‌های تولیدشده روی تمام Mutant ها اجرا می‌شوند و Mutation Score محاسبه می‌شود. اگر هدف حاصل نشود، یک حلقه بازخورد فعال می‌شود. این چرخه ادامه می‌یابد تا به هدف برسیم.

وقتی یک Mutant با وجود تمام تلاش‌ها زنده می‌ماند، دو حالت ممکن است: ضعف تست یا هم‌ارزی واقعی. نکته مهم: مسیر دقیق ارسال به Equivalent Mutant Registry مستقیماً از خروجی Failure Diagnosis & Equivalent Analyzer است.

این پنج ویژگی، موتور اول را از ابزارهای سنتی متمایز می‌کند. اما محدودیت اساسی این موتور، ماهیت Black-box مدل زبانی است که موتور دوم برای پاسخ به آن طراحی شده.

برخلاف موتور اول که حول یک خطلوله فازبه فاز طراحی شده، موتور دوم بر اساس زیرسیستم‌های مجزا و مستقل ساخته شده است. زیرسیستم زیرساخت به صورت Cross-cutting روی تمام بخش‌های دیگر اثر می‌گذارد.

موتور اول، با وجود قدرت بالا در تولید تست خلاقانه، روی یک مدل زبانی Black-box تکیه دارد؛ یعنی نمی‌توانیم با قطعیت ریاضی بگوییم خروجی آن، کمینه‌ترین مجموعه تست ممکن است یا نه.

این زیرسیستم در طرح اولیه‌ها کمتر دیده می‌شد اما در معماری واقعی پروژه نقش حیاتی دارد. موتور دوم از ابتدا برای استفاده تکرارشونده و قابل‌بازتولید در محیط‌های صنعتی طراحی شده است.

علاوه بر عملگرهای استاندارد، پنج عملگر Integration Operators تعریف شده‌اند که مخصوص نقص‌های سطح کنترل برنامه هستند. هر عملگر نیز به یک فهرست از حالت‌های ممکن نگاشت می‌شود.

نظریه Coupling Effect می‌گوید خطاهای پیچیده و واقعی نرم‌افزار، معمولاً ترکیبی از چند خطای ساده‌ترند. بنابراین، اگر مجموعه تست ما بتواند تمام خطاهای مرتبه اول را شناسایی کند، خطاهای پیچیده‌تر را هم ضمناً پوشش خواهد داد.

موتور دوم برای ساخت استخر اولیه تست، از سه استراتژی کاملاً مکمل استفاده می‌کند. ACOC با یک سقف عددی و نمونه‌گیری با بذر ثابت، از انفجار ترکیبیاتی پیشگیری می‌کند و نتایج همیشه تکرارپذیر می‌مانند.

پس از تولید تست توسط هر سه استراتژی، منابع در یک مجموعه واحد ترکیب می‌شوند. یکتاسازی از اجرای بی‌فایده و پرهزینه تست‌های تکراری روی Mutant ها جلوگیری می‌کند.

چارچوب اجرای موتور دوم برای هر اجرا یک پروسه کاملاً مجزا با سه لایه حفاظتی دارد. زمان سنج تطبیقی هوشمند، زمان مجاز هر Mutant را بر اساس زمان اجرای واقعی کد سالم تخمین می‌زند.

خروجی واقعی زیرسیستم اجرا، فقط Kill Matrix نیست، بلکه ترکیبی از KillMatrix به همراه OrigResults است. OrigResults برای تغذیه زمان سنج تطبیقی و تشخیص رفتار متفاوت جهش یافته‌ها لازم است.

مسئله NP-Hard است، یعنی هیچ الگوریتمی با زمان اجرای چندجمله‌ای برای حل دقیق و بهینه آن در حالت کلی شناخته شده نیست. در اسلاید بعد می‌بینیم چگونه این مسئله را عملاً حل می‌کنیم.

برای هر تست، یک متغیر دودویی تعریف می‌کنیم. تابع هدف، کمینه‌سازی مجموع این متغیرها است. اگر CBC بتواند مسئله را حل کند، پاسخ تضمین‌شده بهینه سراسری است — نه یک تخمین، بلکه یک اثبات ریاضی قطعی.

اگر مسئله آنقدر بزرگ باشد که حل‌گر در زمان معقول جواب ندهد، یا اصلاً امکان‌پذیر نباشد، سیستم به الگوریتم Greedy عقب‌نشینی می‌کند. این مکانیزم اطمینان می‌دهد که حتی در مقیاس‌های بسیار بزرگ، سیستم همیشه یک خروجی تولید می‌کند.

پس از حل مسئله بهینه‌سازی، ممکن است هنوز Mutant هایی زنده مانده باشند. در این حالت، مدل زبانی دوباره فراخوانی می‌شود تا ورودی کاملاً تازه‌ای برای دقیقاً همان Mutant های سرسخت بسازد.

Identity Folding کد جهش‌یافته را از نظر ریاضی ساده می‌کند. اگر ساختار حاصل دقیقاً با کد اصلی یکسان شد، سیستم می‌تواند با قطعیت صد درصد ریاضی اثبات کند که این Mutant هم‌ارز است.

برای گزارش‌دهی دقیق، موتور دوم سه نوع امتیاز مجزا محاسبه می‌کند. Equivalent-aware Score که فقط موارد اثبات‌شده را حذف می‌کند، واقع‌گرایانه‌ترین معیار است.

این پنج ویژگی، دقیقاً چیزهایی هستند که موتور اول به دلیل ماهیت Black-box خود نمی‌تواند با همین سطح از قطعیت ارائه دهد — و این، ما را به مهم‌ترین بخش ارائه می‌رساند: رابطه میان این دو موتور.

اسلاید ۳۴ — رابطه دو موتور (مهم‌ترین اسلاید)

این دو موتور رقیب هم نیستند؛ مکمل یکدیگرند. رابطه از سه جهت: اعتبارسنجی (موتور دوم می‌گوید آیا خروجی موتور اول به بهینه نزدیک است)؛ بنچمارک (عدد کمینه‌ای که موتور دوم پیدا می‌کند، معیار سنجش عینی است)؛ و بازخورد بالقوه برای بهبود متقابل.

مهم‌ترین ردیف جدول، بهینگی سراسری است: موتور اول هیچ تضمینی برای آن ندارد، اما موتور دوم آن را به صورت ریاضی تضمین می‌کند. همین تفاوت، دلیل اصلی نیاز به موتور دوم به عنوان ابزار اعتبارسنجی است.

شش مشارکت اصلی این پروژه: چارچوب هوشمند LLM، بازتعریف نقش ACOC، موتور بهینه‌سازی ریاضی، تشخیص هم‌ارزی ترکیبی، اعتبارسنجی متقابل، و معماری مکمل با پتانسیل پژوهشی.

پیش از پایان، خلاصه کل پروژه در یک جمله: هوش مصنوعی را برای بخشی به کار گرفتیم که در آن خلاقیت لازم بود، و ریاضیات دقیق را برای بخشی که در آن اطمینان و قطعیت لازم بود.

از وقتی که برای شنیدن این ارائه گذاشتید سپاسگزارم. خوشحال می‌شوم به سؤالات شما درباره هرکدام از دو موتور، نحوه ترکیب آن‌ها، یا جزئیات فنی فازهای مختلف پاسخ دهم.